Leeds University
School of Computer Science

UNIVERSITY OF LEEDS

Assessed Coursework Coversheet

For use with *individual* assessed work

Student ID Number:	2	0	2	0	5	0	7	5	8
Module Code:	COMP3011								
Module Title:	Web Services and Web Data								
Module Leader:	Ammar Alsalka								
Declared Word Count:	1415								

Please read the following carefully and be accurate in your responses; they are all important:

	Delete as appropriate
By submitting this work I declare it is all my own work, other than where indicated by references. I have not colluded with others, re-submitted past work of my own, submitted any work done by others or by Generative AI unless indicated, or otherwise breached the University academic integrity rules. I understand that any discrepancies between this declaration and the assignment could result in an academic malpractice procedure. Read the full University of Leeds declaration of academic integrity here https://secretariat.leeds.ac.uk/wp-content/uploads/sites/109/2022/12/academic_integrity.pdf	YES
My declared word count is accurate and I have not attempted to mislead. I understand that making a fraudulent statement about word count could result in an academic malpractice procedure, and/or may impact the mark.	YES
I have applied for an extension but have not yet heard back on whether it is granted. I am submitting this paper in the knowledge that I may request to submit a later version when I hear back from the extension team. Markers should be aware that this may not be my final version of the assignment.	NO
I am aware of the GAI category for this assignment (delete opposite to show the correct one), and have adhered to the guidance for that category.	GREEN

Assignments should be submitted in time but will be accepted (with late penalties) up to 14 days after deadline

Late penalties = 5 marks per 24hours late

Contents

Summary 3
Technology stack justification 3
Django 3
Django REST Framework 3
SQLite 3
Bootstrap 5 4
Python-Chess 4
Chart.js 4
System architecture 5
Challenges and solutions 5
Challenge: User Assignment in Imports 5
Testing approach 5
Manual Testing Strategy 5
Edge Cases Tested ..... **Error! Bookmark not defined.**
Future Testing Improvements ..... **Error! Bookmark not defined.**
Limitations 5
Future developments 6
Lessons learned 6
Generative AI declaration 6
Conclusion 7

Technical Report: My Chess Stats

Summary

My Chess Stats is a web application designed to help chess players track, analyze, and visualize their games from Chess.com and Lichess. The application provides secure user authentication, comprehensive game management, bulk import capabilities, and detailed statistical analysis with interactive charts. This report outlines the key design decisions, technology choices, challenges faced, and future development opportunities for the project.

Technology stack justification

Django

I chose Django because of I had already worked with this framework in the past; I helped create the frontend for a book reading habits app (available on my github as BookTracker). This module also taught us some Django, so I felt it was the natural choice. There are also a few technical reasons:

- Batteries-included philosophy: Django provides built-in authentication, ORM, admin interface, and security features out of the box, significantly reducing development time.
- MVT Architecture: The Model-View-Template architecture provides clear separation of concerns, making the codebase maintainable and scalable.
- Security Features: Django's built-in protection against common vulnerabilities (CSRF, XSS, SQL injection) was crucial for handling user data securely.

Django REST Framework

We were encouraged to base our API on REST, and since I was already using Django, choosing DRF is obvious, since it is easy to integrate and more:

- Seamless Integration: DRF integrates natively with Django's ORM and authentication system.
- ViewSets and Routers: The ViewSet class with DefaultRouter allowed me to create RESTful endpoints with minimal code while maintaining consistency.
- Serialization: DRF's serializer system provides robust validation and transformation of complex data types.
- Browsable API: The built-in browsable API interface proved invaluable during development and testing.
- Custom Actions: The ability to add custom actions (like /import/ and /stats/) to ViewSets kept the code organized.

SQLite

This is the included database in Django and since this is a simple project, I felt that I did not need anything more robust.

- Zero Configuration: SQLite requires no setup, making it ideal for development and testing.
- File-based: The entire database is a single file, simplifying version control and deployment.
- Sufficient for Development: With expected user counts under 100 during development, SQLite's performance is more than adequate.

Bootstrap 5

I worked with Bootstrap 5 in the project mentioned previously and felt comfortable in implementing it once more for this project. I considered using a proper frontend framework, but I stayed with Bootstrap for time-and-experience-related reasons.

- Responsive Design: Bootstrap's grid system ensures the application works on desktop, tablet, and mobile devices.
- Component Library: Pre-built components (cards, modals, navigation) accelerated UI development.
- Customization: Easy to override with custom CSS while maintaining consistency.

Python-Chess

This library makes handling chess related data easy and saved me many hours of development. I primarily used it to process the game positions into an SVG file for visualization.

Chart.js

I used Chart.js to handle the rating progression charts.

- Lightweight: Small footprint compared to alternatives like D3.js.
- Simple API: Easy to create beautiful charts with minimal code.
- Responsive: Charts automatically resize with the viewport.
- Open Source: MIT licensed with active community support.

System architecture

The application follows a traditional Django MVT architecture with a RESTful API layer.

Challenges and solutions

Challenge: User Assignment in Imports

Problem: Despite setting the user in the view, some imported games were created with user_id = NULL.

This problem bugged me for many hours, even leveraging AI wasn't much help. When importing the games into the database, their user field was not being set. I expected this to need a complex solution, but, in the end, the problem was between the driver's seat and the wheel. When defining the model, I accidentally hard coded the user field to always be null, since my initial implementation of the app did not have separate users yet.

```
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='chess_games',
        null=False,  #this was set to true
        blank=False,
        db_index=True
    )
```

Testing approach

Manual Testing Strategy

Due to time constraints, I focused on comprehensive manual testing, using it normally and also trying to find ways to "break" the application

Limitations

Current Limitations

1. Database Performance
    - SQLite may become slow with >10,000 games per user
    - No connection pooling for concurrent users
2. Import Speed
    - Large PGN files (>1000 games) can take 30+ seconds
    - No background task processing (blocks the request)
3. Board Visualization
    - Only shows final position, not move-by-move
    - SVG generation can be slow for complex positions
4. Mobile Experience
    - Not tested but likely doesn't work well

Future developments

1. Background Processing
    - Add progress bars with WebSocket updates
2. Enhanced Testing
    - Write unit tests for all models
    - Add integration tests for API endpoints
    - Create test fixtures with sample data
3. Performance Optimization
    - Add database indexes for all filtered fields
    - Implement caching for statistics
4. Export Functionality
    - Export filtered games as CSV
    - Export statistics as PDF reports
    - Download games as PGN

Lessons learned

1. Don’t underestimate the complexity of implementing user registration to an app
2. Always double check the model when dealing with data problems
3. Log absolutely everything

Generative AI declaration

“Generative AI is an Artificial Intelligence (AI) technology that automatically generates content in response to written prompts. As a general principle, at the University of Leeds you can use Gen AI to help you learn but cannot use AI to generate or falsify work. You should always analyse and verify the information Gen AI tools provide, rather than accepting it at face value. Whilst you may be given additional information about how Gen AI can complement your wider learning in this module, the use of Gen AI for this assessment has been categorised as:”

“**GREEN**, which means AI has an integral role, and the use of AI tools is required as part of the assessment. You must also declare how AI was used for the assessment.”

I leveraged the use of AI (Deepseek) throught this assignment in the following ways:

1. Code structure: Generated boilerplate code for Django views, serializers, and templates
2. Debugging: Helped identify issues with user assignment in imports
3. CSS styling: Provided Bootstrap 5 styling templates
4. Documentation: Provided outlines and proofread the different documents used in this assignment.

Conclusion

My Chess Stats successfully delivers a comprehensive chess game tracking and analysis platform. The application demonstrates:

- Robust Architecture: Clear separation of concerns with Django MVT and REST API
- User Security: Proper authentication and data isolation
- Platform Integration: Successful import from both Chess.com and Lichess
- Insightful Analytics: Meaningful statistics with clear visualizations
- Extensible Design: Easy to add features due to modular structure

The project met all core requirements and provided valuable learning experiences in web development, data modeling, and chess data analysis.

Future development will focus on performance optimization, enhanced testing, and advanced features like Stockfish integration. The foundation is solid for building a comprehensive chess analysis platform.